

LD1-A Sparse CIC and Canonical-Convolution Reference Specification

Deterministic flat-node aggregation, sparse stencil scatter, normalization, and HDR diagnostics

mdstats development specification

2026-07-20

Contents

1	Status and scope	1
2	Scientific ownership and citations	2
3	Inputs	2
3.1	Weighted samples	2
3.2	Grid and cell	2
3.3	Canonical kernel	2
4	Sparse-only canonical stencil support	3
5	Deterministic sparse CIC aggregation	3
6	Sparse canonical convolution	4
7	Highest-density-region details	4
8	Public records and functions	5
9	Resource and failure policy	5
10	Error cases	5
11	Validation matrix	5
12	Non-objectives and next gate	6
13	References	6

1 Status and scope

This specification implements architecture gate **LD1-A** for `mdstats 0.19.45a0`. It is subordinate to the *Dynamical Framework and Density Plotting Architecture Standard*.

The gate introduces a deliberately simple sparse numerical oracle:

weighted periodic samples $\rightarrow$ sorted CIC node masses $\rightarrow$ sparse canonical stencil scatter $\rightarrow$ normalized flat-node field.

The representation stores sorted logical flat indices and positive values. It is not the production block-sparse backend. Block packing, partial-block masks, transactional sparse storage planning, and public backend selection belong to LD1-B and LD4.

This gate does not change the default dense backend, the default `legacy_spectral_v1` operator, scene composition, probability-shell meshing, or framework-edge quadrature.

2 Scientific ownership and citations

Periodic trilinear cloud-in-cell assignment follows the particle-mesh construction of Hockney and Eastwood (1988). Highest-density-region thresholds follow Hyndman (1996). The finite-support `discrete_periodized_v1` stencil was specified and implemented in LD0-K. The sparse flat-node representation, deterministic accumulation order, resource policy, residual normalization, and debugging conversion limits are project-specific mdstats definitions.

3 Inputs

3.1 Weighted samples

The only source record is the existing immutable `PeriodicWeightedSamples3D`:

```
PeriodicWeightedSamples3D(
    fractional_positions: float64[n_samples, 3], # folded to [0, 1)
    weights: float64[n_samples], # nonnegative
    source_provenance: DensitySourceProvenance,
    total_measure: float,
    measure_kind: "occupancy" | "arc_length",
    measure_units: str,
    sample_group_ids: int64[n_samples] | None,
)
```

The weights must satisfy

$$\left| \sum_s w_s - M \right| \leq 5 \times 10^{-13} \max(1, M),$$

where M is `total_measure`.

3.2 Grid and cell

The logical shape is

$$\mathbf{N} = (N_1, N_2, N_3), \quad N_i \in \mathbb{N}^+.$$

Logical node (i, j, k) lies at

$$\mathbf{f}_{ijk} = \left(\frac{i}{N_1}, \frac{j}{N_2}, \frac{k}{N_3} \right).$$

The row-vector display cell $H \in \mathbb{R}^{3 \times 3}$ must be finite and nonsingular. Its voxel volume is

$$\Delta V = \frac{|\det H|}{N_1 N_2 N_3}.$$

3.3 Canonical kernel

LD1-A supports only `discrete_periodized_v1`. The Gaussian bandwidth obeys $\sigma \geq 0$ and the kernel-tail tolerance lies in $[10^{-15}, 10^{-3}]$.

4 Sparse-only canonical stencil support

LD0-K stores both a dense logical stencil and its active support. LD1-A adds `PeriodicGaussianStencilSupport`, which stores only:

```
PeriodicGaussianStencilSupport(
    grid_shape,
    display_cell,
    gaussian_bandwidth,
    kernel_tail_tolerance,
    cutoff_radius,
    active_flat_indices,    # sorted, unique int64
    active_weights,         # positive float64, sum exactly one within tolerance
    pre_normalization_sum,
    normalization_factor,
    periodic_image_contribution_count,
    covariance,
    metadata,
)
```

Candidate integer images are enumerated using the reciprocal-plane bounds already specified by LD0-K. Retained image contributions are mapped to canonical periodic flat indices and accumulated in image-enumeration order. The final origin weight receives the residual correction

$$g_0 \leftarrow g_0 + \left(1 - \sum_{\delta} g_{\delta}\right).$$

No logical dense stencil is allocated. A guarded `to_dense_values(max_nodes=...)` helper exists only for bounded testing and debugging.

For $\sigma = 0$, the support is the exact identity:

$$\mathcal{S} = \{0\}, \quad g_0 = 1.$$

5 Deterministic sparse CIC aggregation

For one folded sample $\mathbf{f}_s$, define

$$\mathbf{y}_s = \mathbf{N} \odot \mathbf{f}_s, \quad \mathbf{i}_s = \lfloor \mathbf{y}_s \rfloor, \quad \mathbf{d}_s = \mathbf{y}_s - \mathbf{i}_s.$$

For $\epsilon \in \{0, 1\}^3$, the periodic target node is

$$\mathbf{n}_{s\epsilon} = (\mathbf{i}_s + \epsilon) \bmod \mathbf{N},$$

with contribution

$$m_{s\epsilon} = w_s \prod_{a=1}^3 \begin{cases} 1 - d_{sa}, & \epsilon_a = 0, \\ d_{sa}, & \epsilon_a = 1. \end{cases}$$

Contributions are generated in the fixed order

(ox, oy, oz) lexicographic, then original sample order

and accumulated with unbuffered `float64` additions into sorted unique flat nodes. Zero contributions are omitted. The deposited measure must satisfy

$$\left| \sum_j m_j - M \right| \leq 5 \times 10^{-13} \max(1, M).$$

The output is `SparseCICNodeMasses3D`.

6 Sparse canonical convolution

Let occupied CIC nodes be (j, m_j) and canonical stencil support be (δ, g_δ) . Every pair contributes

$$\widetilde{m}_{(j+\delta) \bmod \mathbf{N}} += m_j g_\delta.$$

Pairs are generated in the fixed order

`stencil flat-index order`, then occupied CIC `flat-index order`

so the addition order tracks the dense direct canonical convolution. Periodic target indices are sorted only for storage; additions retain the declared pair order.

If the raw scattered measure is M_{raw} , apply

$$\beta = \frac{M}{M_{\text{raw}}}, \quad m_g^{\text{norm}} = \beta \widetilde{m}_g.$$

A deterministic residual correction is applied to the first stored node. Density is then

$$\rho_g = \frac{m_g^{\text{norm}}}{\Delta V}.$$

The output is `SparseCanonicalDensityReference3D`, a flat-node reference field that implements the backend-neutral scalar-field and periodic-node-access contracts.

7 Highest-density-region details

For $0 < q < 1$, sort positive stored densities in descending order and choose the first threshold c_q satisfying

$$\Delta V \sum_{\rho_g \geq c_q} \rho_g \geq qM.$$

All exact ties at c_q are included. `SparseHDRDetails` records:

- requested mass fraction;
- threshold;
- achieved mass fraction;
- selected node count;
- exact threshold-tie count;
- selected and total measures.

Implicit zero nodes do not alter a positive threshold for $q < 1$.

8 Public records and functions

```
build_periodic_gaussian_stencil_support(...)
aggregate_periodic_cic_sparse(...)
scatter_periodic_stencil_sparse(...)
prepare_sparse_canonical_density_reference(...)
```

The main records are:

```
PeriodicGaussianStencilSupport
SparseCICNodeMasses3D
SparseHDRDetails
SparseCanonicalDensityReference3D
```

All public arrays are C-contiguous defensive copies, read-only, shape-validated, and finite. Sparse flat indices are strictly increasing and unique.

9 Resource and failure policy

The reference path uses explicit hard limits:

```
max_cic_contributions
max_stencil_candidate_contributions
max_kernel_pairs
max_workspace_bytes
max_nodes for debugging conversion
```

Before allocating contribution or pair vectors, the implementation computes a conservative workspace bound. A request exceeding any limit raises `GraphComplexityError` before the corresponding large allocation.

The reference path may allocate arrays proportional to the CIC contribution count or kernel-pair count. It is therefore a correctness oracle, not the final scalable backend. LD1-B must replace flat pair materialization with block-oriented production storage while retaining LD1-A as a test oracle.

10 Error cases

The implementation rejects:

1. nonfinite or nonfolded weighted samples;
2. negative weights or inconsistent total measure;
3. invalid logical shapes or singular cells;
4. noncanonical smoothing operators;
5. nonpositive sparse stored masses or densities;
6. mismatched CIC and stencil shapes;
7. resource-limit violations;
8. dense debugging conversion beyond `max_nodes`;
9. zero or destroyed measure after deposition or convolution.

11 Validation matrix

Required fixtures are:

Case	Required property
orthogonal off-grid samples	dense-direct agreement
LTA primitive cell	exact Cartesian metric and dense-direct agreement
face crossing	periodic continuity and dense-direct agreement
edge crossing	periodic continuity and dense-direct agreement
corner crossing	periodic continuity and dense-direct agreement

Case	Required property
multiple images in support	canonical image aggregation
overlapping sources	deterministic duplicate aggregation
bimodal hopping	multimodal field and HDR agreement
independent ensemble	arbitrary nonnegative weights
$\sigma = 0$	exact identity convolution

Acceptance against dense direct convolution of `discrete_periodized_v1` is:

<code>relative L1 field error</code>	<code><= 2e-11</code>
<code>relative L-infinity field error</code>	<code><= 5e-11</code>
<code>absolute integral error</code>	<code><= 5e-13 * max(1, total_measure)</code>
<code>HDR threshold absolute difference</code>	<code><= 5e-12 * max(1, reference_maximum)</code>
<code>achieved HDR mass-fraction difference</code>	<code><= 5e-13</code>

Additional gates require:

- repeated identical inputs produce byte-identical sparse indices, values, and metadata;
- periodic integer translations agree within the field tolerances;
- sparse CIC agrees with dense CIC within 2×10^{-16} absolute node mass;
- sparse-only stencil support agrees with the LD0-K dense stencil within 2×10^{-16} absolute weight;
- every resource limit is exercised by a focused failure test;
- the unchanged dense legacy path remains compatible with `mdstats 0.19.44a0`.

12 Non-objectives and next gate

LD1-A does not expose `grid_backend="local_sparse"` through production plotting options. It does not pack blocks, serialize block fields, render sparse clouds, or extract sparse meshes.

The next gate is **LD1-B**, which will implement production atomic block packing, partial-block masks, transactional sparse preflight, public node access, structured serialization, and the localized LTA storage benchmark.

13 References

1. Hockney, R. W., and J. W. Eastwood. *Computer Simulation Using Particles*. Taylor & Francis, 1988. Periodic particle-mesh cloud-in-cell assignment.
2. Hyndman, R. J. "Computing and Graphing Highest Density Regions." *The American Statistician* **50** (1996): 120-126. DOI: 10.1080/00031305.1996.10474359.